<- This means this lab is on the hive

Lab 2: C Debugging

61C Summer 26

Please do this lab in order according to the spec

- Exercises 2, 3, 4 are on the same file and if you do more than the instructions ask you to do for any of the exercises it might mess up the autograder

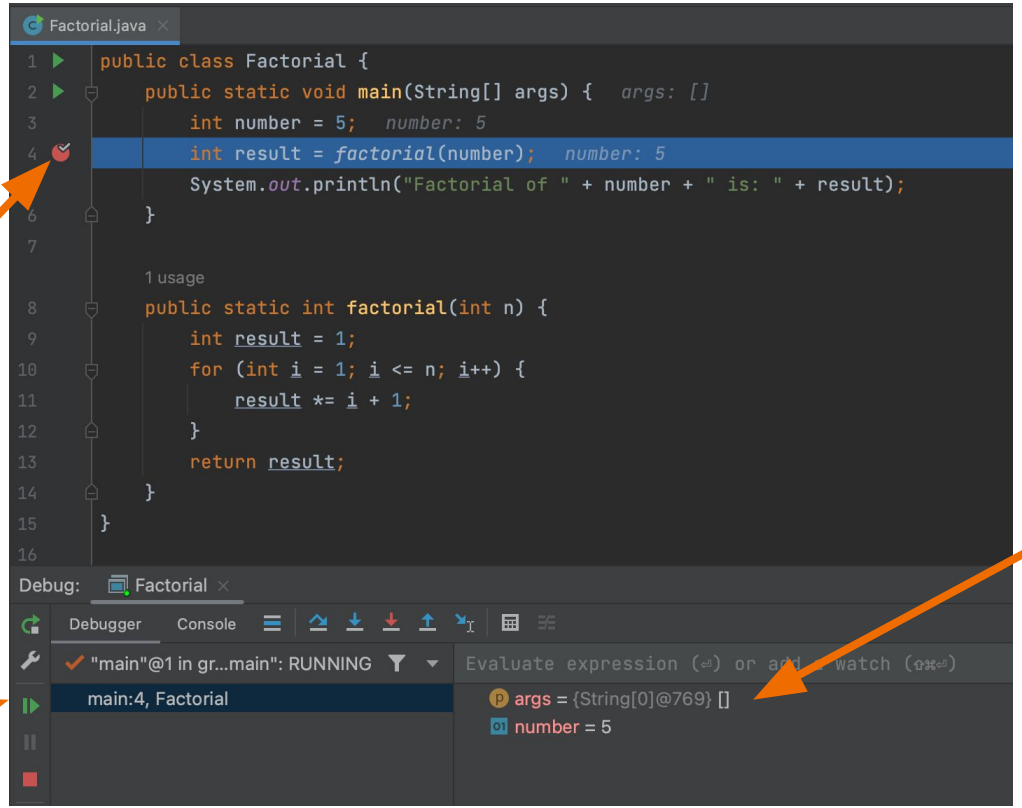
Compiler Warnings

- Compiler warnings are generated to help you find potential bugs in your code. Your code will still compile.
- Make sure that you fix all of your compiler warnings before you attempt to run your code
  - This will save you a lot of time debugging in the future because fixing the compiler warnings is much faster than trying to find the bug on your own.
- Compiler warning will catch
  - Implicit type conversion, uninitialized variables, unused variables, ...
- Compiler errors (won't compile) will catch:
  - Syntax errors, undeclared, type mismatch, ...

GDB

- GDB, the GNU Project debugger, allows you to see what is going on 'inside' another program while it executes -- or what another program was doing at the moment it crashed.
- In this class, we will be using CGDB which provides a lightweight interface to gdb to make it easier to use.
- If you want to use GDB, make sure to use the -g flag when compiling:
    - gcc -g example.c

CS 61B Review



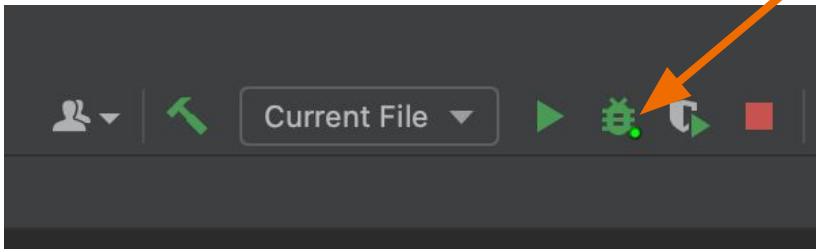
Breakpoint

Variables currently in scope

Continue until next breakpoint

CS 61B Review

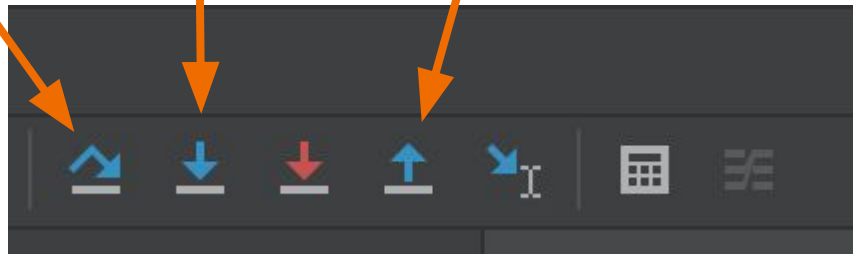
Start debugging



Step over

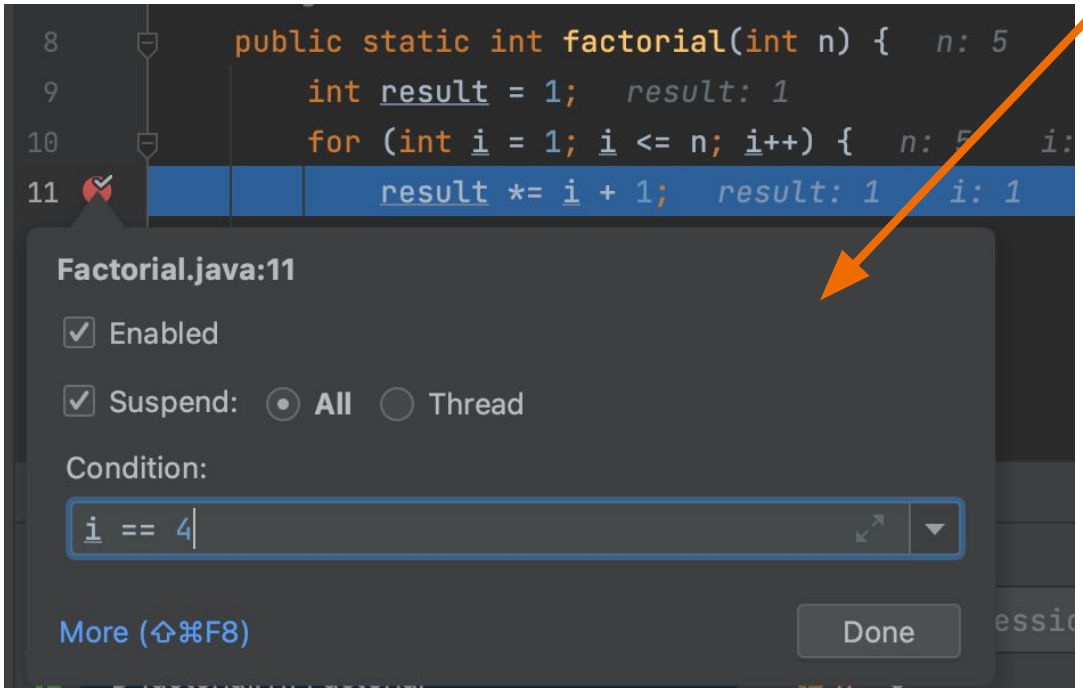
Step into

Step out



CS 61B Review

Conditional breakpoint



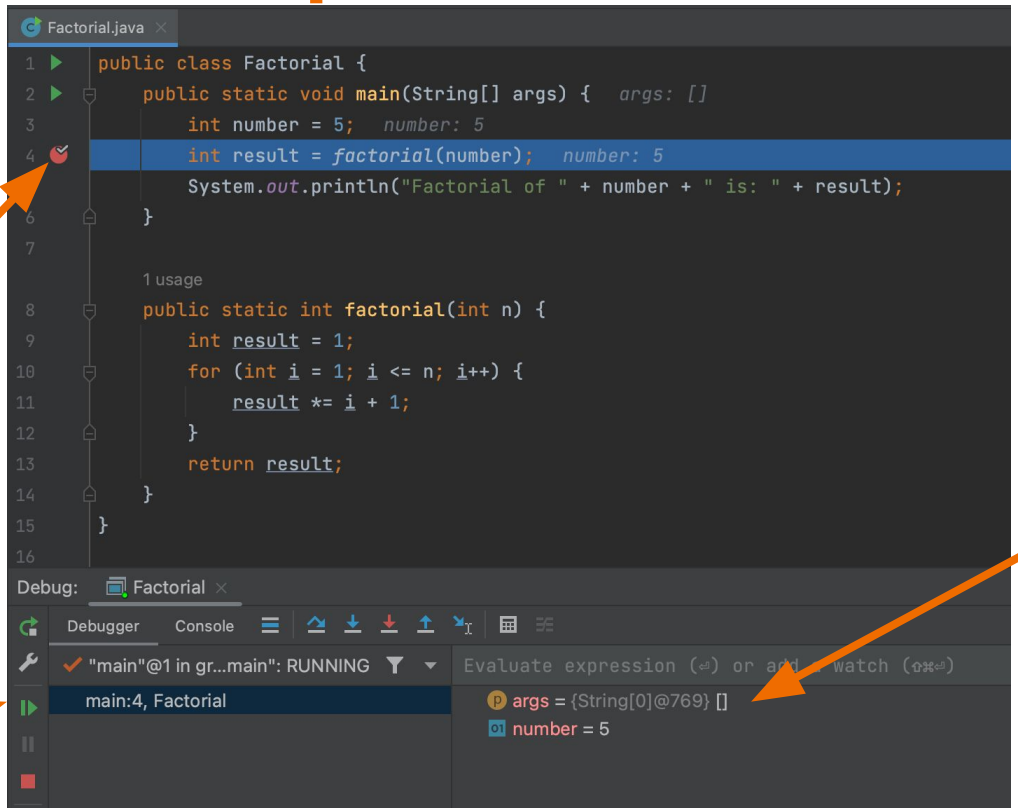
CS 61C Debugger (GDB)

Command	Abbreviation	Description
start	N/A	begin running the program and stop at line 1 in main
step	s	execute the current line of code (this command will step into functions)
next	n	execute the current line of code (this command will not step into functions)
finish	fin	executes the remainder of the current function and returns to the calling function
print [arg]	p	prints the value of the argument
quit	q	exits gdb

Command	Abbreviation	Description
break [line num or function name]	b	set a breakpoint at the specified location, use `filename.c:linenum` to set a breakpoint in a specific file
conditional break (ex: break 3 if n==4)	(ex: b 3 if n==4)	set a breakpoint at the specified location only if a given condition is met
run	r	execute the program until termination or reaching a breakpoint
continue	c	continues the execution of a program that was paused

GDB command equivalents

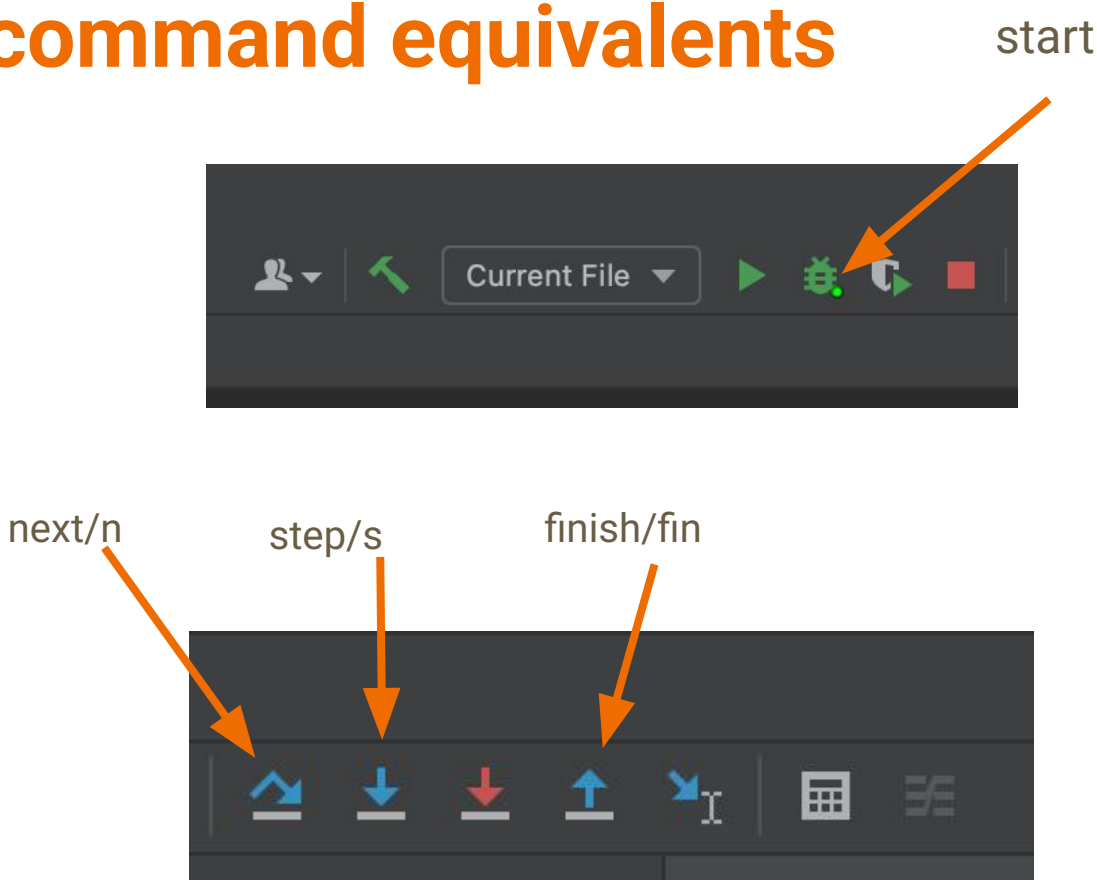
break/b



print/p (roughly speaking)

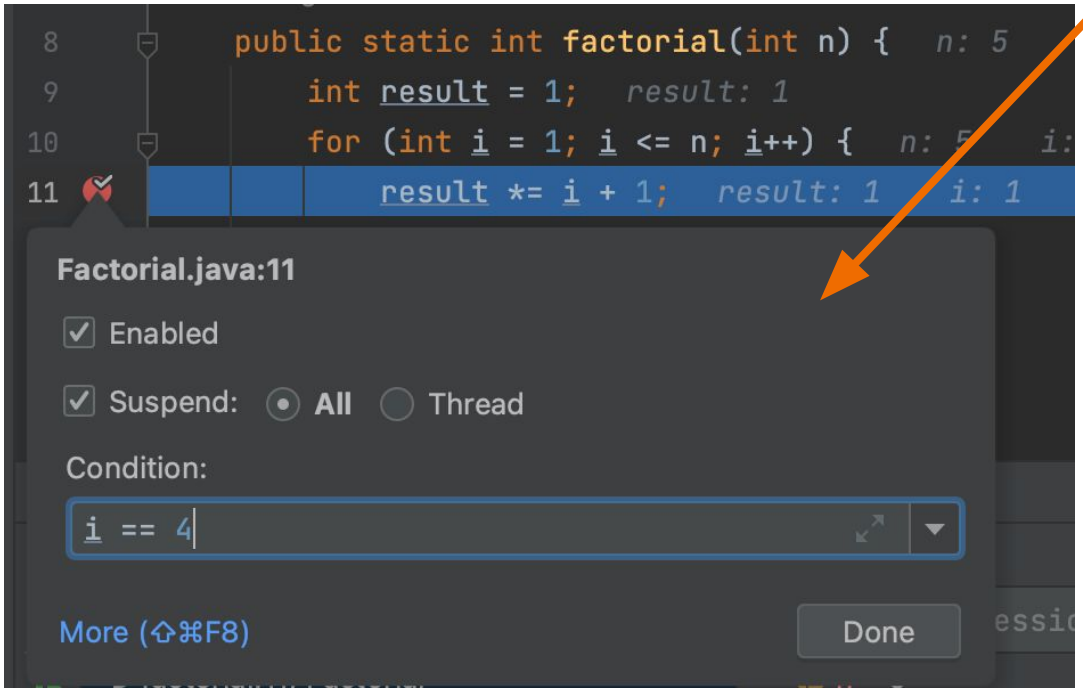
continue/c

GDB command equivalents



GDB command equivalents

b 11 if i == 4



CS 61C Debugger (GDB)

```
~
 1 #include <stdio.h>
 2
 3 int factorial(int n);
 4
 5 int main() {
 6     int number = 5;
 7     int result = factorial(number);
 8     printf("Factorial of %d is: %d\n", number, result);
 9     return 0;
10 }
11
12 int factorial(int n) {
13     int result = 1;
14     for (int i = 1; i <= n; i++) {
15         result *= i + 1;
16     }
17     return result;
18 }
19
~
~
```

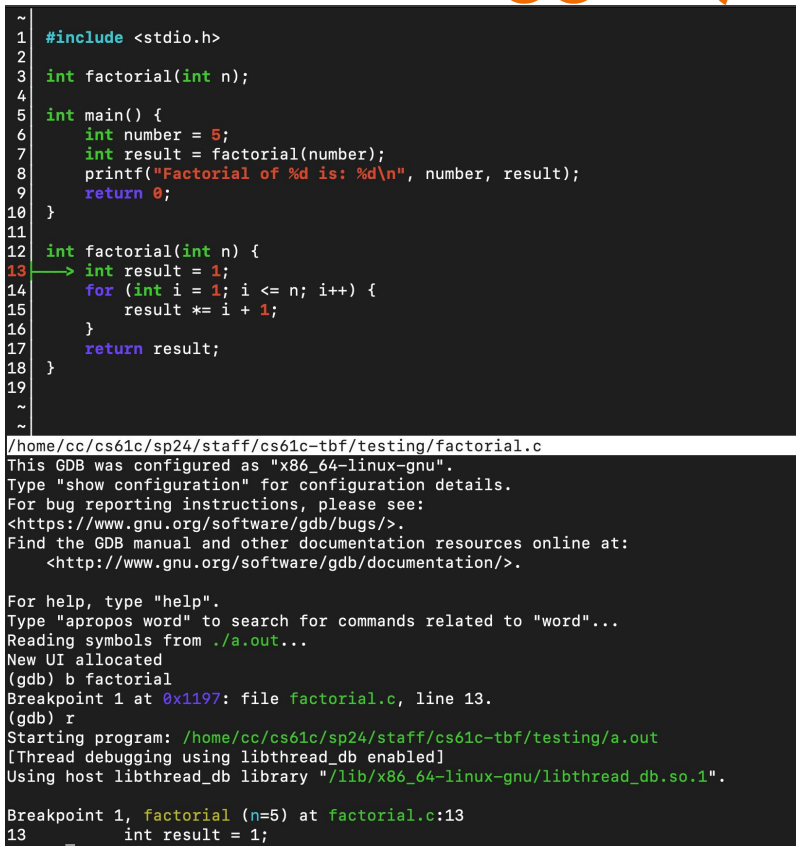
```
/home/cc/cs61c/sp24/staff/cs61c-tbf/testing/factorial.c
GNU gdb (Ubuntu 12.1-0ubuntu1~22.04) 12.1
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./a.out...
New UI allocated
(gdb)
```

Goals:

- Set a breakpoint at the factorial function
- Set a breakpoint at line 15 when i == 4
- Find the bug by checking outputs

CS 61C Debugger (GDB)



```
~
 1  #include <stdio.h>
 2
 3  int factorial(int n);
 4
 5  int main() {
 6      int number = 5;
 7      int result = factorial(number);
 8      printf("Factorial of %d is: %d\n", number, result);
 9      return 0;
10  }
11
12  int factorial(int n) {
13  ---> int result = 1;
14      for (int i = 1; i <= n; i++) {
15          result *= i + 1;
16      }
17      return result;
18  }
19
 ~
 ~
/home/cc/cs61c/sp24/staff/cs61c-tbf/testing/factorial.c
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./a.out...
New UI allocated
(gdb) b factorial
Breakpoint 1 at 0x1197: file factorial.c, line 13.
(gdb) r
Starting program: /home/cc/cs61c/sp24/staff/cs61c-tbf/testing/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, factorial (n=5) at factorial.c:13
13          int result = 1;
```

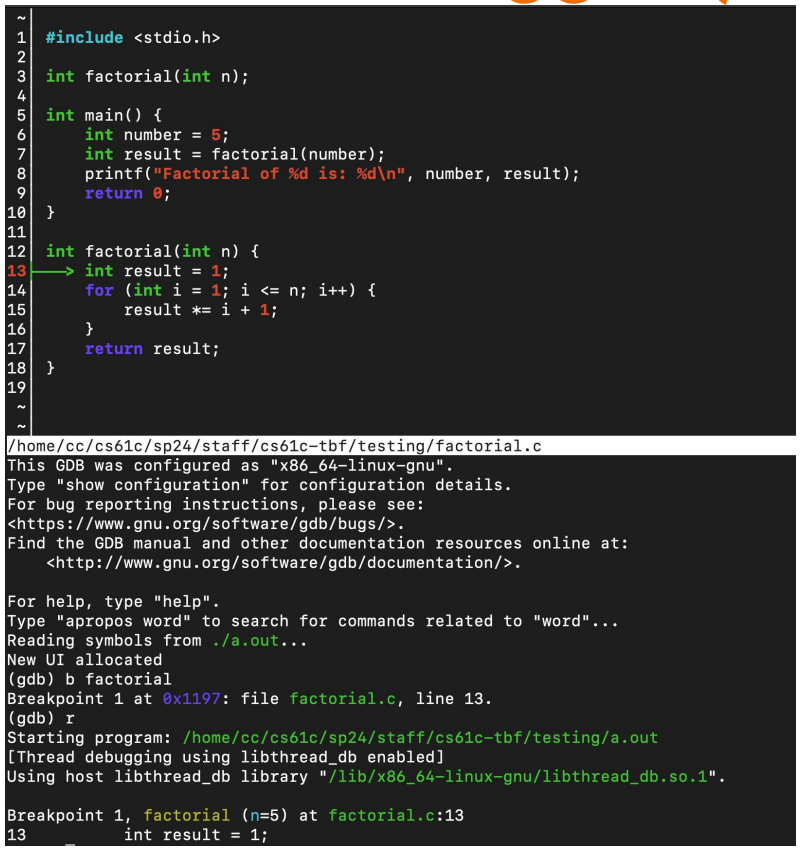
Typed so far:

- b factorial (set a breakpoint at the factorial function)
- r (starts executing the program)

Questions:

- What will p number give me?
- What will p n give me?
- (p prints out the argument in the console)

CS 61C Debugger (GDB)



```
~
 1  #include <stdio.h>
 2
 3  int factorial(int n);
 4
 5  int main() {
 6      int number = 5;
 7      int result = factorial(number);
 8      printf("Factorial of %d is: %d\n", number, result);
 9      return 0;
10  }
11
12  int factorial(int n) {
13 ---> int result = 1;
14      for (int i = 1; i <= n; i++) {
15          result *= i + 1;
16      }
17      return result;
18  }
19
 ~
 ~
/home/cc/cs61c/sp24/staff/cs61c-tbf/testing/factorial.c
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./a.out...
New UI allocated
(gdb) b factorial
Breakpoint 1 at 0x1197: file factorial.c, line 13.
(gdb) r
Starting program: /home/cc/cs61c/sp24/staff/cs61c-tbf/testing/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, factorial (n=5) at factorial.c:13
13          int result = 1;
```

Typed so far:

- b factorial (set a breakpoint at the factorial function)
- r (starts executing the program)

Questions:

- What will p number give me?
  - “No symbol “number” in current context.”
- What will p n give me?
  - 5

CS 61C Debugger (GDB)

```
~
 1  #include <stdio.h>
 2
 3  int factorial(int n);
 4
 5  int main() {
 6      int number = 5;
 7      int result = factorial(number);
 8      printf("Factorial of %d is: %d\n", number, result);
 9      return 0;
10  }
11
12  int factorial(int n) {
13  ---> int result = 1;
14      for (int i = 1; i <= n; i++) {
15          result *= i + 1;
16      }
17      return result;
18  }
19
 ~
 ~
/home/cc/cs61c/sp24/staff/cs61c-tbf/testing/factorial.c
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./a.out...
New UI allocated
(gdb) b factorial
Breakpoint 1 at 0x1197: file factorial.c, line 13.
(gdb) r
Starting program: /home/cc/cs61c/sp24/staff/cs61c-tbf/testing/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, factorial (n=5) at factorial.c:13
13          int result = 1;
```

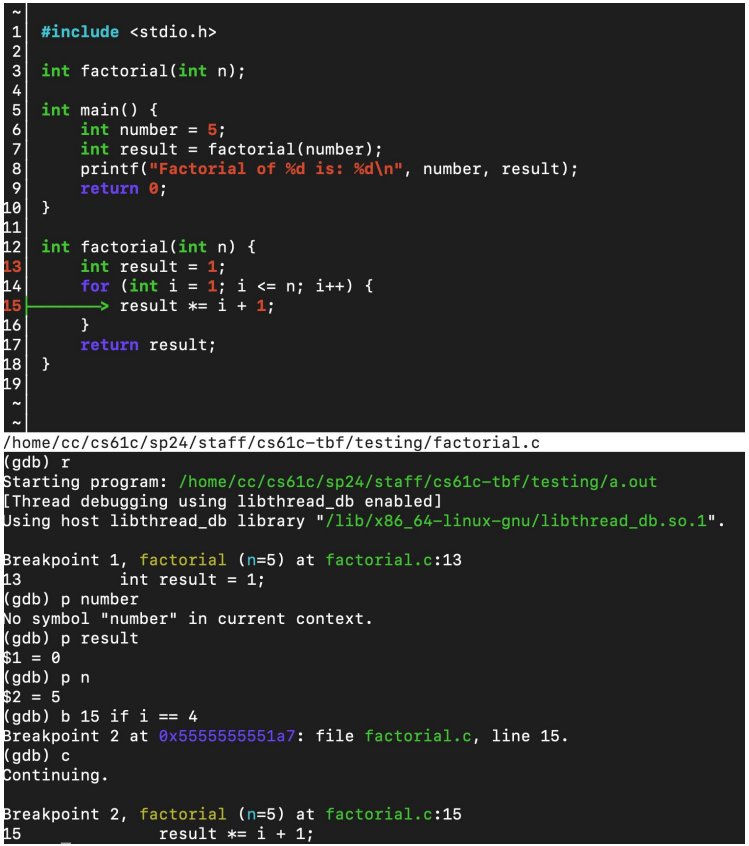
Typed so far:

- b factorial (set a breakpoint at the factorial function)
- r (starts executing the program)

Questions:

- How would I set a breakpoint at line 15 if i == 4 then continue to that point?

CS 61C Debugger (GDB)



```
~
 1  #include <stdio.h>
 2
 3  int factorial(int n);
 4
 5  int main() {
 6      int number = 5;
 7      int result = factorial(number);
 8      printf("Factorial of %d is: %d\n", number, result);
 9      return 0;
10  }
11
12  int factorial(int n) {
13      int result = 1;
14      for (int i = 1; i <= n; i++) {
15  ------> result *= i + 1;
16      }
17      return result;
18  }
19
~
~
/home/cc/cs61c/sp24/staff/cs61c-tbf/testing/factorial.c
(gdb) r
Starting program: /home/cc/cs61c/sp24/staff/cs61c-tbf/testing/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, factorial (n=5) at factorial.c:13
13          int result = 1;
(gdb) p number
No symbol "number" in current context.
(gdb) p result
$1 = 0
(gdb) p n
$2 = 5
(gdb) b 15 if i == 4
Breakpoint 2 at 0x5555555551a7: file factorial.c, line 15.
(gdb) c
Continuing.

Breakpoint 2, factorial (n=5) at factorial.c:15
15              result *= i + 1;
```

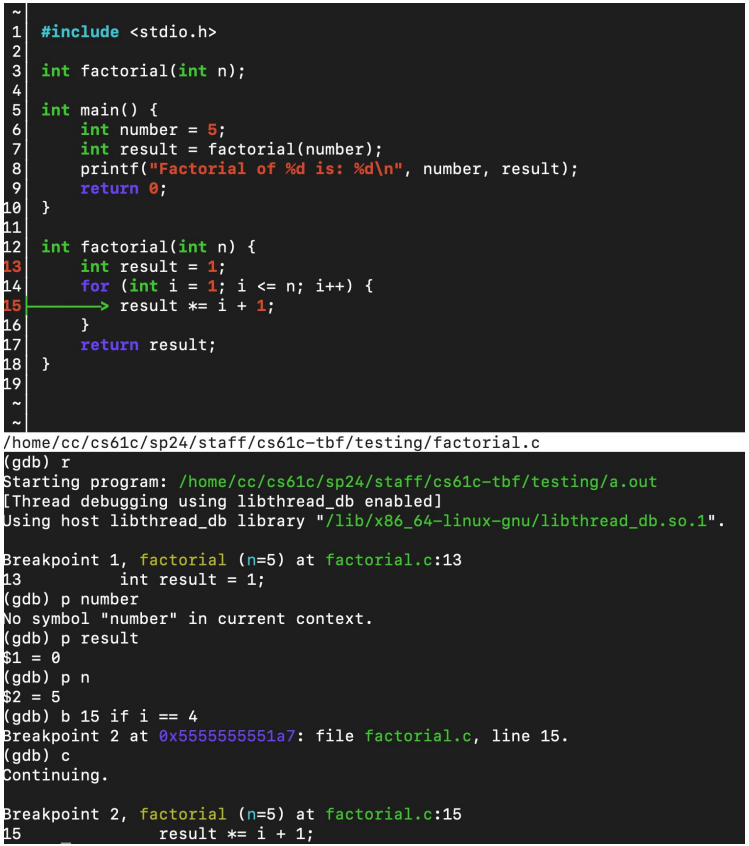
Typed so far:

- b factorial (set a breakpoint at the factorial function)
- r (starts executing the program)
- b 15 if i == 4
- c

Questions:

- After all iterations, what would you expect the value of `result` to be if factorial was implemented correctly?
- How could we check what `result` is at the end of the for loop?

CS 61C Debugger (GDB)



```
~
 1  #include <stdio.h>
 2
 3  int factorial(int n);
 4
 5  int main() {
 6      int number = 5;
 7      int result = factorial(number);
 8      printf("Factorial of %d is: %d\n", number, result);
 9      return 0;
10  }
11
12  int factorial(int n) {
13      int result = 1;
14      for (int i = 1; i <= n; i++) {
15  ------> result *= i + 1;
16      }
17      return result;
18  }
19
~
~
/home/cc/cs61c/sp24/staff/cs61c-tbf/testing/factorial.c
(gdb) r
Starting program: /home/cc/cs61c/sp24/staff/cs61c-tbf/testing/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, factorial (n=5) at factorial.c:13
13          int result = 1;
(gdb) p number
No symbol "number" in current context.
(gdb) p result
$1 = 0
(gdb) p n
$2 = 5
(gdb) b 15 if i == 4
Breakpoint 2 at 0x5555555551a7: file factorial.c, line 15.
(gdb) c
Continuing.

Breakpoint 2, factorial (n=5) at factorial.c:15
15              result *= i + 1;
```

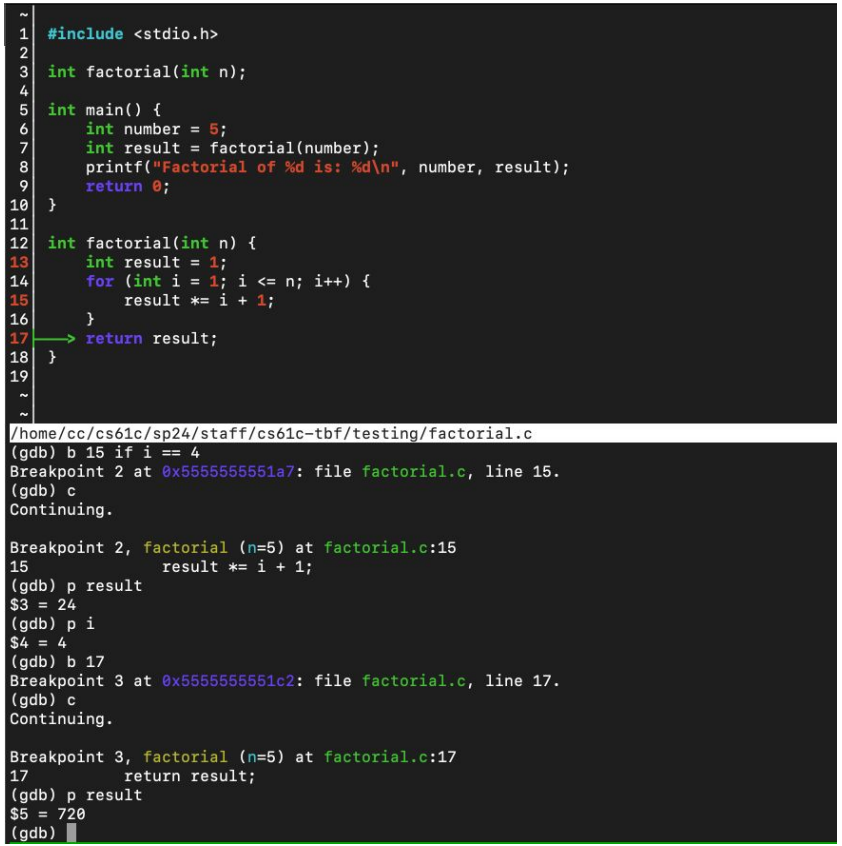
Typed so far:

- b factorial (set a breakpoint at the factorial function)
- r (starts executing the program)
- b 15 if i == 4
- c

Questions:

- After all iterations, what would you expect the value of `result` to be if factorial was implemented correctly?
  - 120
- How could we check what `result` is at the end of the for loop?
  - b 17

CS 61C Debugger (GDB)



```
~
1  #include <stdio.h>
2
3  int factorial(int n);
4
5  int main() {
6      int number = 5;
7      int result = factorial(number);
8      printf("Factorial of %d is: %d\n", number, result);
9      return 0;
10 }
11
12 int factorial(int n) {
13     int result = 1;
14     for (int i = 1; i <= n; i++) {
15         result *= i + 1;
16     }
17 ---> return result;
18 }
19
~
~
/home/cc/cs61c/sp24/staff/cs61c-tbf/testing/factorial.c
(gdb) b 15 if i == 4
Breakpoint 2 at 0x5555555551a7: file factorial.c, line 15.
(gdb) c
Continuing.

Breakpoint 2, factorial (n=5) at factorial.c:15
15              result *= i + 1;
(gdb) p result
$3 = 24
(gdb) p i
$4 = 4
(gdb) b 17
Breakpoint 3 at 0x5555555551c2: file factorial.c, line 17.
(gdb) c
Continuing.

Breakpoint 3, factorial (n=5) at factorial.c:17
17          return result;
(gdb) p result
$5 = 720
(gdb)
```

Typed so far:

- b factorial (set a breakpoint at the factorial function)
- r (starts executing the program)
- b 15 if i == 4
- c
- b 17
- c
- p result
    - This gives us 720

From here, you might want to try using gdb to look at intermediate steps instead of just the end result to find the root cause of the bug.

Some Notes

- You can repeat some commands with numbers
  - If you wanna go 3 lines ahead you can do n 3
- The print feature is very versatile
  - It can print strings, numbers, pointers, etc

Dynamic Memory

- Pointers can be allocated using malloc that persist in the heap
- Heap is a memory space separate from other variables declared in the stack
- Always remove dynamically allocated pointers by calling free(ptr); after use

```
char str1[] = "hello";
char * str2 = malloc(sizeof(char) * 6);
//malloc needs to know how many bytes to allocate
strcpy(str2, str1);

printf ("str1: %s\nstr2: %s\n", str1, str2); //str1: hello
                                             //str2: hello
free(str2);
return strlen(str2); //Crashes. Do not use a pointer after freeing it!
```

Dynamic Memory

Options for allocating memory on the heap:

- **Malloc**: Standard allocation of bytes
  - `malloc(amount of bytes)`, usually `malloc(sizeof(type)*number of items)`
- **Calloc**: Allocate some number of blocks (of given size) and zero out the memory
  - `calloc(number of items, size of each item)`
- **Realloc**: Reallocate pointer to have a new size
  - `realloc(old pointer, new size)`

Stack

```
int main() {
    int* a_ptr = a();
    return 0;
}

int* a() {
    int a = 3;
    return &a;
}
```


main frame

Stack

```
int main() {
    int* a_ptr = a();
    return 0;
}

int* a() {
    int a = 3;
    return &a;
}
```


main frame
a frame

Stack

```
int main() {
    int* a_ptr = a();
    return 0;
}

int* a() {
    int a = 3;
    return &a;
}
```


main frame	
a frame	a = 3

Stack

```
int main() {
    int* a_ptr = a();
    return 0;
}

int* a() {
    int a = 3;
    return &a;
}
```


main frame

Once we return from a, the stack frame for a is popped off the stack, so now our stack only contains `main`.

Effectively, now `a_ptr` is essentially a garbage value because it will be overwritten by future function calls

Heap

```
int main() {
    int* a_ptr = a();

    // do something with a_ptr

    free(a_ptr);

    return 0;
}

int* a() {
    int* a = malloc(sizeof(int));
    *a = 3;
    return a;
}
```


We can think of the heap as a huge pool of memory that persists during program runtime

This is not the same heap as the heap in 61B

Heap

```
int main() {
    int* a_ptr = a();

    // do something with a_ptr

    free(a_ptr);

    return 0;
}

int* a() {
    int* a = malloc(sizeof(int));
    *a = 3;
    return a;
}
```


Heap

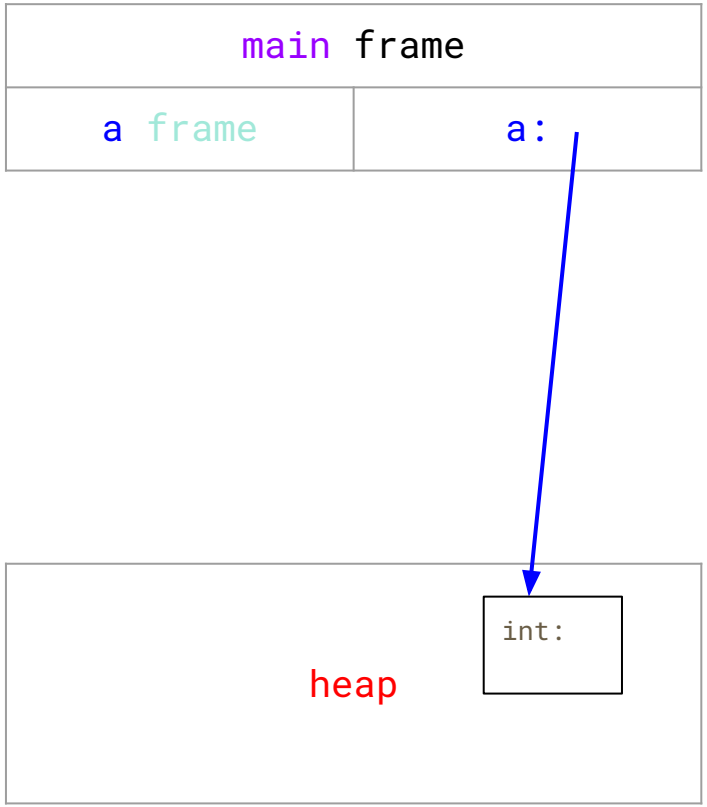
```c
int main() {
    int* a_ptr = a();

    // do something with a_ptr

    free(a_ptr);

    return 0;
}

int* a() {
    int* a = malloc(sizeof(int));
    *a = 3;
    return a;
}
```



Heap

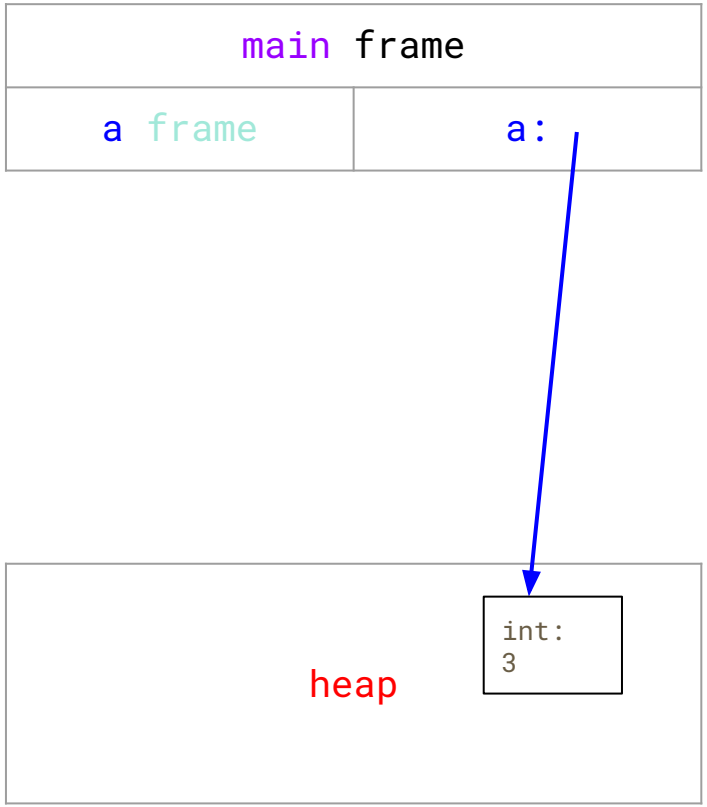
```c
int main() {
    int* a_ptr = a();

    // do something with a_ptr

    free(a_ptr);

    return 0;
}

int* a() {
    int* a = malloc(sizeof(int));
    *a = 3;
    return a;
}
```



Heap

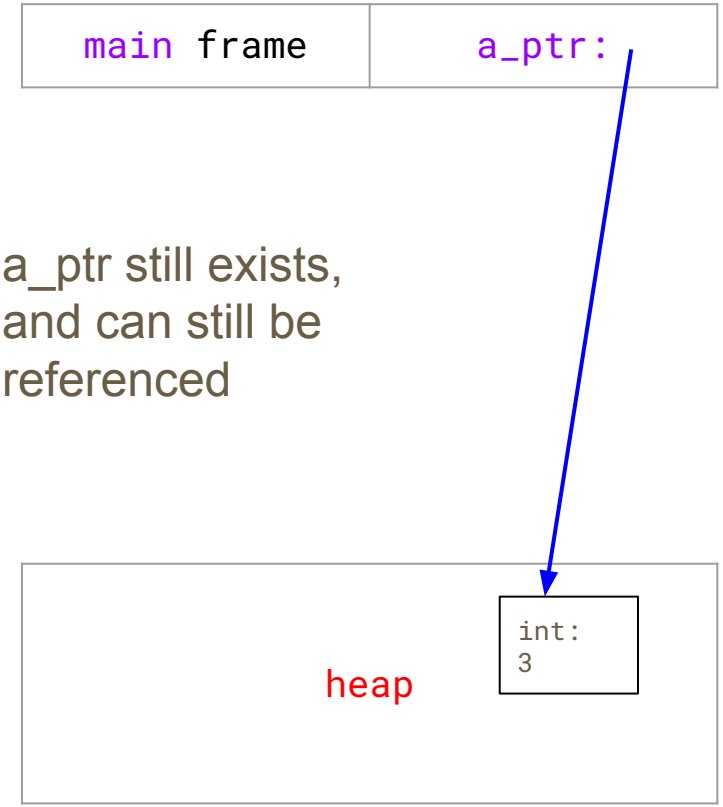
```c
int main() {
    int* a_ptr = a();

    // do something with a_ptr

    free(a_ptr); // REMEMBER TO FREE!

    return 0;
}

int* a() {
    int* a = malloc(sizeof(int));
    *a = 3;
    return a;
}
```

a_ptr still exists, and can still be referenced

Pointers to Structs

- Pass a pointer of a struct to a function so that you can edit the contents
- Access the struct contents by:
  - (*student).major
  - student->major

```
major: chemistry
major: biology
```

```c
#include <stdio.h>
#include <string.h>

typedef struct {
    char first_name[50];
    char last_name[50];
    char major[50];
    int age;
} Student;

void update_major (Student *student, char *new_major) {
    //strcpy((*student).major, new_major);
    strcpy(student->major, new_major);
}

int main() {
    Student s1;
    strcpy(s1.major, "chemistry");
    printf("major: %s\n", s1.major);
    update_major(&s1, "biology");
    printf("major: %s\n", s1.major);
}
```

Arrow operator

Valgrind

- A tool for debugging “bohrbugs” and “heisenbugs”
- Allows you to see invalid reads/writes
- Checks for memory leaks
  - --leak-check=full
- Follow along with the bork.c example!